Microprocessor Finals Preparation Dialog

MBSD
PDF

Microproce...'s Archive
PDF

ASSIGNMENT...1 & 2 MBSD
DOCX

time for Microprocessor based system design finals
so this is the book from which teacher teach us and the course for finals are
Microcontroller and Microprocessor Difference
PPI- Programmable Peripheral Interface
Handshaking
Programmable Interrupt controller
I/O data Transfer
DMA
Program Data Transfer Conditional and Unconditional
Double Buffering Solution
Control Word Register (PPI)
3 byte Instruction
Assignment
some projects block diagram (Air quality index, weather forecasting)

i have attached Past Paper (MBSD pdf) too , for you to analyze the teachers pattern and important thing, teacher loves book language , she like key words and long answers , not just simple definitions, she wants block diagram and long explanations and book language used.
and as we are 5th sem junior year student so you know final is not gonna be piece of cake

dont have to answer all at once , take your time and chats but give me long answers + book wording + block diagrams + exam ready answers

we will do past paper first , so starting by solving question 1

Based on the **Microprocessors and Programmed Logic** book (specifically Chapter 8.7) and the **MBSD Past Paper** you uploaded, here is the comprehensive, exam-ready answer for **Question 1**.

This response uses the specific "book language" and technical keywords your teacher looks for, focusing on the **practical design differences** between the two modes of the USART (Universal Synchronous/Asynchronous Receiver/Transmitter), likely referring to the **Intel 8251A** which matches your 8085 curriculum.

Question 1 Solution: Practical Differences in USART Design for Synchronous vs. Asynchronous Communication

Answer Strategy: To answer this "Level-6" (Design/Create) question, we must look beyond simple definitions and explain how the *system design* and *interfacing* change when configuring the USART for these two distinct modes.

1. Introduction to the USART

The **USART (Universal Synchronous/Asynchronous Receiver/Transmitter)** is a programmable LSI (Large Scale Integration) communications interface designed to provide serial data communications in microprocessor systems. It is capable of operating in two distinct modes, depending on the system requirements for speed and distance.

- **Book Definition:** The USART is a "digital communications subsystem capable of receiving serial data and providing a parallel output and/or of receiving parallel data and providing a serial output." It operates in **full duplex**, meaning both receiving and transmitting can occur simultaneously.

---

2. Designing for Asynchronous Communication

Book Concept: Asynchronous transfer is used for "low speed, low data rate transfers" (typically 110 to 9600 baud) where data appears in bursts with idle time between characters.

Practical Design & Operation: In this mode, the system design relies on **framing** rather than a shared clock signal.

- **Framing:** The design must account for the overhead of a **Start Bit** (logic 0) to synchronize the receiver, followed by data bits, an optional parity bit, and **Stop Bits** (logic 1) to mark the end of the character.
- **Clocking (The Critical Difference):** The practical difference here is that the transmitter and receiver do not share a physical clock wire. Instead, they operate on independent internal clocks that must be set to the same **Baud Rate**.
- **Sampling:** To ensure accuracy without a shared clock, the USART receiver samples the line at a rate much faster than the baud rate (typically **16x or 64x** the baud rate) to locate the center of the start bit and avoid errors caused by signal distortion.

Block Diagram Description: In the block diagram below, notice the **Baud Rate Generator** feeding the TxC and RxC inputs. The data is framed with Start/Stop bits.

---

3. Designing for Synchronous Communication

Book Concept: Synchronous transmission is used for "high speed blocks of data" where efficiency is paramount. In this mode, "one character is followed immediately by another" without start/stop bits.

Practical Design & Operation: The system design changes drastically to support continuous streams of data.

- **Synchronization (SYNC Characters):** Instead of start/stop bits, the design uses specific **SYNC characters** (e.g., ASCII SYN). The receiver enters a "Hunt Mode" to search the bit stream for this specific bit pattern to establish character boundaries.
- **Clocking:** A practical design requirement is that the **transmitter clock must be transmitted to the receiver**, or the receiver must use a Phase Locked Loop (PLL) to

recover the clock from the data stream. The clock rate is **1x** the data rate (unlike the 16x/64x in asynchronous).

- **Data Stream:** If the microprocessor cannot provide data fast enough to keep the stream continuous, the USART must automatically insert SYNC characters (a process called "padding") to maintain synchronization.

Block Diagram Description: In the synchronous setup, notice the **Common Clock** line shared between Transmitter and Receiver, and the absence of Start/Stop bit generation logic in favor of SYNC detection logic.

---

4. Practical Differences Summary (Exam Table)

To satisfy the "practical differences" part of the question, use this comparison which highlights the *design implications*:

Feature	**Asynchronous Design**	**Synchronous Design**
Synchronization	**Bit-level:** Uses a **Start Bit** (0) to trigger synchronization and **Stop Bits** (1) to reset.	**Block-level:** Uses **SYNC Characters** (1 or 2 bytes) at the start of a message block.
Clock Connection	**No physical clock connection** is required between devices. Tx and Rx use independent local clocks (Baud Rate Generators).	**Physical clock connection required** (or clock recovery circuit). The Tx clock is sent to the Rx to ensure perfect lock.
Efficiency & Overhead	**Low Efficiency:** 20-30% of transmission time is wasted on Start/Stop bits (e.g., 10 bits sent for 8 bits of data).	**High Efficiency:** No Start/Stop bits. Overhead is only the initial SYNC bytes; ideal for large blocks of data.
Clock Rate (x)	Requires **16x or 64x** clock inputs to allow the receiver to sample and find the center of the bit.	Requires **1x** clock input (Data Rate = Clock Rate) as bits are perfectly synchronized.
Idle Line	The line can be **idle (Mark/High)** for any duration between characters.	The line **cannot be idle** during a frame. If data stops, the USART must insert "SYNC" characters automatically.
Error Detection	Checks for **Framing Errors** (missing Stop bit) and **Overrun Errors**.	Checks for **CRC** (Cyclic Redundancy Check) or Parity Errors; framing errors do not exist.

Teacher's "Book Language" Checklist:

- **Keywords used:** Baud Rate, Framing Error, SYNC Character, Hunt Mode, Full Duplex, Mark/Space, Start/Stop Bits, 16x/64x Sampling.
- **Citation style:** Definitions grounded in *Microprocessors and Programmed Logic* (Short).
- **Depth:** Explains *why* the design changes (e.g., sampling rates, clock lines) rather than just listing features.

Would you like me to proceed to the next question from the Past Paper or the Assignment?

Your uploads may be too large for the best results. Learn more

now solve question 2 both a and b part

Based on **Chapter 8** of the *Microprocessors and Programmed Logic* book (Short), here is the detailed, exam-ready solution for **Question 2 (a) and (b)**.

This answer strictly adheres to the "book language" and "long explanation" requirement, utilizing the specific diagrams and definitions found in Sections **8.7** and **8.6.2**.

Question 2(a): Interface for Serial Transfer

Prompt: Construct a block diagram that shows the interface between microprocessor and I/O device for serial transfer of information. Explain the functions provided by the interface to support serial data transfer.

1. Block Diagram Construction

According to **Figure 8.7-1** in the text, the interface between a microprocessor (which operates in parallel) and a serial I/O device requires two distinct stages: a **Logical Interface** and an **Electrical Interface**.

[Block Diagram Description]

- **Left Block: Microcomputer** (Parallel Data)
- **Middle Block 1: Logical Interface** (Handles Serial/Parallel Conversion & Formatting)
- **Middle Block 2: Electrical Interface** (Handles Signal Level Translation)
- **Right Block: Serial I/O Device** (e.g., Teletype, CRT)
- **Connections:** A single communication channel (or pair for full duplex) connects the Electrical Interface to the I/O Device.

(Note: In your exam, draw the blocks linearly as described above. Label the arrows between the Microcomputer and Logical Interface as "Parallel Data" and the arrows between the Electrical Interface and I/O device as "Serial Signal").

2. Functions Provided by the Interface

To support serial data transfer, the interface must provide two primary functions to bridge the gap between the microprocessor's parallel architecture and the device's serial nature:

- **Logical Formatting (Parallel-to-Serial / Serial-to-Parallel Conversion):** The microprocessor transfers data on the data bus in parallel (all bits simultaneously). However, the I/O device (like a Teletype) requires data one bit at a time.
  - **Transmitting:** The interface must accept a parallel word from the microprocessor and convert it into a serial stream of bits. This often involves adding **framing bits** (Start Bit, Stop Bits) and **Parity bits** for error checking.
  - **Receiving:** The interface must receive a serial stream, remove the framing/parity bits (logical formatting), and assemble the data bits into a parallel word that the microprocessor can read.

    - **Synchronization:** The interface must establish a common **Baud Rate** (bits per second) to ensure the receiver samples the data at the correct time intervals.
- **Electrical Signal Translation:** The microprocessor operates on standard **TTL logic levels** (e.g., +5V for Logic 1, 0V for Logic 0). However, serial I/O devices and communications channels often require different voltage or current levels to transmit data over distances.
    - **Function:** The electrical interface translates the logic signals from the microprocessor into the electrical signals appropriate for the communications channel (e.g., RS-232 voltage levels or 20mA current loops).

Question 2(b): 8255A Programmable Peripheral Interface Modes

Prompt: Illustrate the different modes of operation provided by the 8255A Programmable Peripheral Interface that can be used by the microprocessor.

Introduction

The **8255A Programmable Peripheral Interface (PPI)** is a general-purpose LSI I/O device. It contains three 8-bit ports (A, B, and C) and is configured by writing a **Control Word** into its control register. The 8255A provides three distinct modes of operation.

Mode 0: Basic Input-Output

This mode provides simple input and output operations for each of the ports. No "handshaking" is required; data is simply written to or read from a specified port.

- **Configuration:** It provides two 8-bit ports (A and B) and two 4-bit ports (C-upper and C-lower).
- **Operation:**
    - Any port can be programmed independently as **Input** or **Output**.
    - **Outputs are latched**, meaning the data remains on the pins until changed.
    - **Inputs are not latched**, meaning the microprocessor reads the instantaneous value at the pins.
- **Application:** Used for simple devices like switches, LEDs, or relays where synchronization (handshaking) is not necessary.

Mode 1: Strobed Input-Output

This mode allows for the transfer of data that is synchronized by handshaking signals. This is critical when the speed of the I/O device differs from the microprocessor.

- **Configuration:** Ports A and B function as 8-bit data ports. Port C bits are used as **control signals** (handshaking) for Ports A and B.
- **Operation:**
    - **Input (Strobed Input):** The input device places data on the port and sends a **Strobe (STB)** signal. The 8255A latches the data and sends an **Input Buffer Full (IBF)** signal back to the device and an **Interrupt Request (INTR)** to the microprocessor.
    - **Output (Strobed Output):** The microprocessor writes data to the port. The 8255A sends an **Output Buffer Full (OBF)** signal to the device. When the device accepts the data, it sends an **Acknowledge (ACK)** signal.

Mode 2: Bidirectional Bus

This mode is designed for communicating with devices that transmit and receive data over a single 8-bit bus (like a microprocessor-to-microprocessor interface).

- **Configuration:** This mode uses **Port A** only. Port A becomes a single 8-bit **bidirectional bus**.
- **Operation:**
  - Five bits of Port C are used for status and control (handshaking) for Port A.
  - This allows Port A to both transmit and receive data, using handshaking signals to control the flow direction and synchronization.
  - While Port A is in Mode 2, Port B can still be used in Mode 0 or Mode 1.

(Exam Tip: For a "Level-3" question, ensuring you distinguish that Mode 0 is "Basic," Mode 1 is "Strobed/Handshake," and Mode 2 is "Bidirectional" is the key to full marks.)

question 3 a, b and c part

Based on the **MBSD Past Paper** and the instructional style of the **Microprocessors and Programmed Logic** book (Short), here is the detailed solution for **Question 3 (a), (b), and (c)**.

I have corrected the OCR errors in the assembly code for Part (C) based on standard 8085 mnemonics (e.g., `INC SKIP` → `JNC SKIP`).

Question 3(a): Groups of Instructions by Functionality

Prompt: Explain the different types of groups of instructions according to their functionality.

Answer: In the design of the 8085 microprocessor, the instruction set is classified into five functional groups. Each group allows the microprocessor to perform specific types of operations on data.

1. Data Transfer Group This group includes instructions that move data between registers, or between memory and registers. These instructions **copy** data from a source to a destination without modifying the source.

- **Function:** They facilitate the loading of constants, copying data, and accessing memory. Crucially, these instructions generally **do not affect the Condition Flags** (except for `POP PSW`).
- **Examples:**
  - `MOV A, B` (Move content of Register B to Accumulator)
  - `MVI A, 05H` (Move Immediate 05H to Accumulator)
  - `LHLD 2000H` (Load H and L registers direct from memory 2000H)

2. Arithmetic Group These instructions perform arithmetic operations on the data in the registers and memory.

- **Function:** They support addition, subtraction, incrementing, and decrementing. The results are typically stored in the **Accumulator**, and these operations **affect the Condition Flags** (Zero, Carry, Sign, Parity, Auxiliary Carry) based on the result.

- **Examples:**
  - `ADD B` (Add Register B to Accumulator)
  - `ADI 05H` (Add Immediate 05H to Accumulator)
  - `DAD D` (Double Add; Add DE pair to HL pair)

3. Logical Group This group performs logical (Boolean) operations on data in the Accumulator and registers/memory.

- **Function:** It includes AND, OR, XOR, Compare, Rotate, and Complement operations. Like arithmetic instructions, these **affect the flags** to reflect the result (e.g., Zero flag set if result is 0).
- **Examples:**
  - `ANA B` (Logical AND Register B with Accumulator)
  - `ORA M` (Logical OR Memory content with Accumulator)
  - `RAR` (Rotate Accumulator Right through Carry)

4. Branch Group These instructions allow the microprocessor to change the sequence of program execution, either unconditionally or based on the status of the flags.

- **Function:** They alter the **Program Counter (PC)** to jump to a different memory address. This is essential for loops (`JNZ`), decision making (`JC`), and subroutine calls (`CALL`).
- **Examples:**
  - `JMP 2050H` (Unconditional Jump to address 2050H)
  - `JZ 2050H` (Jump if Zero flag is set)
  - `CALL 3000H` (Call subroutine at 3000H)

5. Stack, I/O, and Machine Control Group This group manages the Stack, performs Input/Output operations, and controls the CPU state.

- **Function:** Used for maintaining the stack pointer, communicating with peripheral ports, and halting the processor or handling interrupts.
- **Examples:**
  - `PUSH B` (Push BC pair onto Stack)
  - `IN 00H` (Input data from Port 00H)
  - `HLT` (Halt execution)

Question 3(b): Assembly Program (16-bit Addition)

Prompt: Develop a program in assembly language that adds two 16-bit numbers from different memory locations and store the result in some other memory location.

Explanation: To add 16-bit numbers efficiently in the 8085, we use the **HL register pair** as an accumulator for 16-bit data and the **DAD** (Double Add) instruction.

- **Number 1 Location:** 2000H (LSB), 2001H (MSB)
- **Number 2 Location:** 2002H (LSB), 2003H (MSB)
- **Result Location:** 2004H (LSB), 2005H (MSB)

Assembly Code:

Code snippet

```
; -- 16-bit Addition Program --
LHLD 2000H      ; Load HL pair with 1st 16-bit number from memory [2000H-2001H]
XCHG            ; Exchange HL with DE. Now 1st number is in DE pair.
LHLD 2002H      ; Load HL pair with 2nd 16-bit number from memory [2002H-2003H]
DAD D           ; Add DE to HL (HL = HL + DE). Result is now in HL.
SHLD 2004H      ; Store HL (the 16-bit result) to memory location [2004H-2005H]
HLT             ; Terminate program
```

Memory Map Example:

- If `2000H` has `01` and `2001H` has `02` (Number = 0201H)
- If `2002H` has `03` and `2003H` has `04` (Number = 0403H)
- Result in `2004H` will be `04` and `2005H` will be `06` (Result = 0604H).

Question 3(c): Trace Assembly Program

Prompt: Write step by step the function performed by the following assembly language program. *(Note: I have corrected OCR typos such as `INC SKIP` to `JNC SKIP` and `DOR` to `DCR` to make the code functional).*

Reconstructed Code:

Code snippet

```
MVI B, 60H    ; Initialize Register B with 60H
MVI C, 05H    ; Initialize Loop Counter C with 5
MOV A, D      ; Move data from D to Accumulator (Assuming 'ALD' was 'A,D')
BACK: RAR     ; Rotate Accumulator Right through Carry
JNC SKIP      ; Jump to SKIP if No Carry (Carry = 0)
INR B         ; Increment B (Only happens if Carry = 1)
SKIP: DCR C   ; Decrement Counter C
JNZ BACK      ; Jump to BACK if C is not zero
HLT           ; Halt
```

Step-by-Step Function Analysis: This program analyzes the bit pattern of the data in **Register D**. Specifically, it checks the **5 Least Significant Bits (LSBs)** of the data.

1. **Initialization:** It sets a base value of **60H** in Register B and sets a loop counter of **5** in Register C. It loads the data to be tested from D into the Accumulator.
2. **Rotation (The Test):** inside the loop, the `RAR` instruction rotates the Accumulator right. The Least Significant Bit (LSB) enters the **Carry Flag**.
3. **Conditional Increment:**
   - The `JNC SKIP` instruction checks the Carry Flag.
   - If the bit shifted out was a **'1'** (Carry Set), the jump is **not** taken, and `INR B` is executed (B increments).
   - If the bit shifted out was a **'0'** (Carry Reset), the program **jumps** to SKIP, bypassing the increment.
4. **Looping:** The loop runs 5 times (`DCR C`), effectively scanning the 5 LSBs of the data.

5. **Final Result:**
    - Register **B** will contain **60H + (Number of 1s in the lower 5 bits of D)**.

Summary Statement for Exam: "The program counts the number of logic **1s** in the 5 least significant bits of the data in Register D and adds this count to an initial offset of 60H, storing the final result in Register B."

question 4

Based on **Chapter 3** (Section 3.3.2 and Table 3.3-1) of **Kenneth L. Short's** *Microprocessors and Programmed Logic*, here is the comprehensive, exam-ready answer for **Question 4**.

This answer utilizes the specific "book language" and describes the **State Transition Diagram** (Figure 3.3-6) required by your teacher.

Question 4: Activities Associated with T-States and State Transition Diagram

Answer Strategy: To answer this "Level-2" question, you must define the T-state in relation to the system clock, draw (or describe) the **State Transition Diagram** found on **page 92** of the textbook, and list the specific hardware activities that occur in each state as detailed in **Table 3.3-1**.

1. Introduction: T-States and Machine Cycles

The operation of the 8085A microprocessor is cyclical and synchronized by the system clock.

- **T-State:** A T-state is defined as "one clock period in duration." It is the basic unit of time for the microprocessor.
- **Machine Cycle:** The fetching and execution of a single instruction consists of one or more machine cycles. Each machine cycle consists of **three to six T-states** (T_1 to T_6).

2. The State Transition Diagram

*(Note for Exam: You must draw **Figure 3.3-6** from page 92 of the book. Description follows below for you to reproduce it.)*

Diagram Description to Draw:

1. Draw circles representing states labeled $T_1, T_2, T_3, T_4, T_5, T_6$.
2. **Transitions:**
    - Draw an arrow from $T_1 \rightarrow T_2$.
    - Draw an arrow from $T_2 \rightarrow T_3$.
    - From T_3, draw two arrows: one going back to T_1 (End of Cycle) and one going to T_4.
    - From T_4, draw two arrows: one going back to T_1 (End of Cycle) and one going to T_5.
    - From T_5, draw two arrows: one going back to T_1 and one going to T_6.
    - From T_6, draw an arrow going back to T_1.

Explanation of the Diagram: As shown in the state transition diagram, each machine cycle always starts in state T_1. The microprocessor sequentially traverses states T_2 and T_3.

Depending on the specific instruction and machine cycle type (e.g., OPCODE FETCH vs. MEMORY READ), the microprocessor may terminate the cycle after T_3 and return to T_1 of the next cycle, or proceed to states T_4, T_5, and T_6 to complete internal processing.

---

3. Activities Associated with T-States (Table 3.3-1)

The specific hardware activities that occur during each state are crucial for this answer.

State T_1: This is the address setup state.

- **Address:** The microprocessor places the **memory or I/O device address** on the bus. The high-order byte resides on the Address Bus ($A_8 - A_{15}$) and the low-order byte resides on the multiplexed Address/Data Bus ($AD_0 - AD_7$).
- **ALE:** An **Address Latch Enable (ALE)** pulse is generated. This facilitates the latching of the low-order address bits from $AD_0 - AD_7$ into an external latch.
- **Status:** Status information is placed on the **IO/$\overline{M}$**, **S1**, and **S0** lines to define the *type* of machine cycle (e.g., Opcode Fetch, Memory Read).
- **Flags/Interrupts:** The **HALT flag** is checked, and the **READY** and **HOLD** inputs are sampled.

State T_2: This state prepares the bus for data transfer.

- **Ready/Hold:** The **READY** and **HOLD** inputs are sampled again. If READY is low, the microprocessor may enter a **WAIT state** (T_{WAIT}) after T_2 to synchronize with slow memory.
- **Program Counter:** The **Program Counter (PC)** is incremented if the machine cycle is part of an instruction fetch.
- **Bus Float:** The multiplexed Address/Data bus ($AD_0 - AD_7$) is floated (put in high impedance) to allow the external memory/device to drive the bus for a read operation.

State T_3: This is the data transfer state.

- **Data Transfer:** An instruction byte (Opcode) or data byte is transferred **to/from the microprocessor**.
- **Control Signal:** The $\overline{RD}$ (Read) or $\overline{WR}$ (Write) control strobe is active during this state to enable the transfer.

State T_4: This state is primarily for decoding and decision making.

- **Decoding:** During an **OPCODE FETCH** machine cycle, the contents of the **Instruction Register (IR)** are decoded.
- **Decision:** The processor determines whether to enter state T_5 or to terminate the machine cycle and enter T_1 of the next cycle.

States T_5 and T_6:

- **Execution:** These states are used to complete the execution of complex instructions that require internal processing time beyond the fetch and decode phases.

Summary Checklist for Exam Score:

1. **Definitions:** Did you define T-state (1 clock period) and Machine Cycle?

2. **Diagram:** Did you draw the bubble diagram showing transitions $T_1 \rightarrow T_2 \rightarrow T_3 \ldots$?
3. **T1 Details:** Did you mention **ALE** and **Status signals**?
4. **T2 Details:** Did you mention **PC Increment**?
5. **T3 Details:** Did you mention **Data Transfer**?
6. **T4 Details:** Did you mention **Decoding**?

Would you like me to proceed to Question 5 (PIC 8259A)?

question 5

Based on the **MBSD Past Paper** context and standard microprocessor texts (like **Kenneth L. Short's** *Microprocessors and Programmed Logic*, Chapter 9), here is the comprehensive, exam-ready answer for **Question 5**.

This question focuses on the **8259A Programmable Interrupt Controller (PIC)**, a critical device in 8085A systems for managing complex interrupt environments.

Question 5: 8259A Programmable Interrupt Controller

Prompt: Explain the operation of the 8259A Programmable Interrupt Controller, focusing on its internal block diagram, priority arbitration, vector generation, and modes of operation.

1. Introduction

The **Intel 8259A** is a Programmable Interrupt Controller (PIC) designed to work with the 8085A and 8086 microprocessors. It functions as an overall manager in an interrupt-driven system, accepting requests from up to **eight** peripheral sources ($IR_0 - IR_7$), resolving priority collisions, and supplying a programmed vector address to the processor to branch to the appropriate service routine.

2. Internal Block Diagram & Functional Blocks

To answer the "Internal Block Diagram" part of the question, you must describe the data flow through the specific registers.

The internal architecture consists of the following key blocks:

- **Interrupt Request Register (IRR):** This 8-bit register stores all interrupt levels that are currently requesting service. The lines $IR_0 - IR_7$ are connected directly to this register.
- **Interrupt Mask Register (IMR):** This register stores the bits which mask the interrupt lines. If a bit is set (logic 1), the corresponding interrupt line (IR_n) is masked (disabled).
- **In-Service Register (ISR):** This register stores the priority levels that are currently being serviced. A bit is set in this register when the processor acknowledges an interrupt.
- **Priority Resolver:** This logic block determines the highest priority among the bits set in the IRR. It also checks the ISR to ensure the new request has a higher priority than the routine currently executing (if any).
- **Data Bus Buffer:** This 3-state, bidirectional 8-bit buffer ($D_0 - D_7$) interfaces the 8259A to the system data bus. Control words and status information are transferred through this

buffer.

- **Read/Write Logic:** Accepts control signals ($\overline{RD}, \overline{WR}, A_0, \overline{CS}$) from the CPU to control the transfer of data.
- **Cascade Buffer/Comparator:** This block stores and compares the IDs of all 8259As in a system. It allows multiple 8259As to be chained (cascaded) to expand the interrupts up to 64.

3. Priority Arbitration

The 8259A performs "Priority Arbitration" to handle simultaneous interrupts.

- **Fixed Priority (Fully Nested Mode):** By default, IR_0 has the highest priority and IR_7 has the lowest.
- **The Sequence:**
  1. Interrupt requests arrive at the **IRR**.
  2. The **Priority Resolver** compares these requests against the **IMR** (to ignore masked interrupts) and the **ISR** (to respect current service routines).
  3. If the request is valid and has higher priority than any interrupt currently in the ISR, the 8259A asserts its **INT** (Interrupt) output to the CPU.

4. Vector Generation (The Handshake)

Once the 8085A receives the INT signal, it responds with **Interrupt Acknowledge ($\overline{INTA}$)** pulses. The 8259A uses these pulses to "vector" the microprocessor to the correct subroutine.

- **1st $\overline{INTA}$ Pulse:** The 8259A resolves the highest priority interrupt and freezes the state of the interrupts. It places the **CALL instruction opcode (CDH)** onto the data bus. The 8085A reads this opcode.
- **2nd $\overline{INTA}$ Pulse:** The 8259A places the **low-order byte** of the interrupt vector address onto the data bus.
- **3rd $\overline{INTA}$ Pulse:** The 8259A places the **high-order byte** of the interrupt vector address onto the data bus.

The 8085A then executes this `CALL` instruction, pushing the Program Counter (PC) to the stack and jumping to the vector address provided by the 8259A.

5. Modes of Operation

The 8259A is highly programmable via **Initialization Command Words (ICWs)** and **Operation Command Words (OCWs)**.

- **Fully Nested Mode:** The default mode. Priorities are ordered 0 to 7. An interrupt request is acknowledged only if it is higher priority than the one currently in service.
- **Rotating Priority Modes:**
  - *Automatic Rotation:* After a peripheral is serviced, it is assigned the lowest priority. This ensures all devices get equal turns.
  - *Specific Rotation:* The programmer specifies which level has the lowest priority.
- **Poll Mode:** The internal interrupts are disabled. The microprocessor polls the 8259A (reads a status word) to check if an interrupt is pending.

- **Masked Mode:** Individual interrupts can be enabled or disabled by writing to the **IMR**.
- **End of Interrupt (EOI):**
  - *Normal EOI:* The command is issued by the CPU at the end of the service routine to reset the corresponding bit in the ISR.
  - *Automatic EOI:* The ISR bit is reset automatically at the end of the 3rd $\overline{INTA}$ pulse.

Exam Tip: For a "Level-6" (Analysis/Design) score, ensure you explicitly mention the **three $\overline{INTA}$ pulses** in the Vector Generation section, as this is the specific handshake mechanism between the 8259A and the 8085A.